COMPUTER ENGINEERING

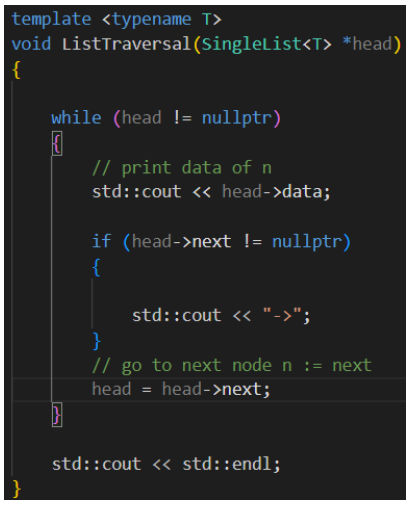
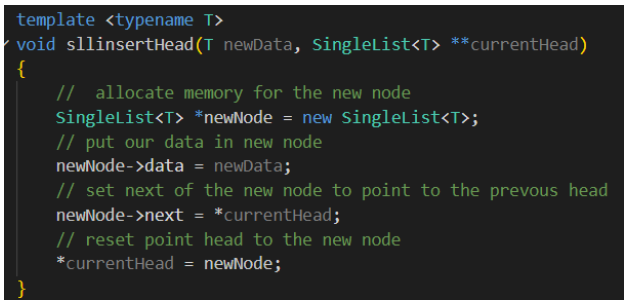
QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 07-29-2026 | **DATE SUBMITTED:** | 07-29-2026 |
| **NAME:** | GUARIN, JAN MIKAELA C. | **INSTRUCTOR:** | ENGR. JIMLORD QUEJADO |

## ACTIVITY NO. 4.1 Array-Based vs. STL Stack Implementations

### 1. PROCEDURE OUTPUT

What procedures did you perform?

| Operation | Screenshot |
|---|---|
| **Traversal** |  |
| **Insertion at head** |  |

```
template <typename T>
void ListTraversal(SingleList<T> *head)
{

    while (head != nullptr)
    {
        // print data of n
        std::cout << head->data;

        if (head->next != nullptr)
        {

            std::cout << "->";
        }
        // go to next node n := next
        head = head->next;
    }

    std::cout << std::endl;
}
```

```
template <typename T>
void sllinsertHead(T newData, SingleList<T> **currentHead)
{
    //  allocate memory for the new node
    SingleList<T> *newNode = new SingleList<T>;
    // put our data in new node
    newNode->data = newData;
    // set next of the new node to point to the prevous head
    newNode->next = *currentHead;
    // reset point head to the new node
    *currentHead = newNode;
}
```

**Insertion at any part of the list**

```
template <typename T>
void sllGeneralInsert(T newData, SingleList<T> *prevNode)
{
    // 1. Check if it is the head node (previous node is null)
    if (prevNode == nullptr)
    {
        // 2. If null, print "Previous node cannot be null."
        std::cout << "Previous value cannot be null" << std::endl;
        return;
    }

    // 3. Allocate a new node
    SingleList<T> *newNode = new SingleList<T>;

    // 4. Store data in the new node
    newNode->data = newData;

    // 5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    // 6. Point previous node to the new node
    prevNode->next = newNode;
}
```

**Insertion at the end**

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T> **head)
{
    // 1. Allocate a new node
    SingleList<T> *newNode = new SingleList<T>;
    // 2. Dereference to the head node
    SingleList<T> *currentNode = *head;
    // 3. Store data in new node
    newNode->data = newData;
    // 4. Point to Null
    newNode->next = nullptr;

    // 5. Traverse the list until next of the node is null

    while (currentNode->next != nullptr)
    {
        currentNode = currentNode->next;
    }

    // 6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

| | |
|---|---|
| **Deletion of node** | |

```
template <typename T>
void sllDeleteNode(T findData, SingleList<T> **head)
{

    // if there is nothing to be deleted.
    if (*head == nullptr)
        return;

    SingleList<T> *currNode = *head;
    SingleList<T> *prevNode = nullptr;

    // searching the node: 1. the node doesnt exist + we found data
    while (currNode != nullptr && currNode->data != findData)
    {
        prevNode = currNode;
        currNode = currNode->next;
    }

    // if data not found currNode == nullptr in the while loop:
    if (currNode == nullptr)
    {
        std::cout << "the data is not found \n"
                  << std::endl;
        return;
    }

    // if data was found
    if (prevNode == nullptr)
    {
        *head = currNode->next;
    }
    else
    {
        prevNode->next = currNode->next;
    }
    delete currNode;
}
```

Table 3-2. Code for the List Operations

| | | |
|---|---|---|
| a. | **Source Code** | `std::cout << "Testing of Traversal: \n";`<br>`ListTraversal(head);` |
| | **Console** | `Testing of Traversal:`<br>`M->I->K->A` |
| b. | **Source Code** | `std::cout << "Testing of sllinsertHead: \n";`<br>`sllinsertHead('J', &head);`<br>`ListTraversal(head);` |
| | **Console** | `Testing of sllinsertHead:`<br>`J->M->I->K->A` |
| c. | **Source Code** | `std::cout << "Testing of sllGeneralInsert: \n";`<br>`sllGeneralInsert('L', head->next->next);`<br>`ListTraversal(head);` |

| | | |
|---|---|---|
| | **Console** | `Testing of sllGeneralInsert:`<br>`J->M->I->L->K->A` |
| d. | **Source Code** | `std::cout << "Testing of sllinsertEnd: \n";`<br>`sllinsertEnd('C', &head);`<br>`ListTraversal(head);` |
| | **Console** | `Testing of sllinsertEnd:`<br>`J->M->I->L->K->A->C` |
| e. | **Source Code** | `std::cout << "Testing of deleting the node: \n";`<br>`sllDeleteNode('J', &head);`<br>`ListTraversal(head);`<br>`sllDeleteNode('L', &head);`<br>`ListTraversal(head);`<br>`sllDeleteNode('C', &head);`<br>`ListTraversal(head);` |
| | **Console** | `Testing of deleting the node:`<br>`M->I->L->K->A->C`<br>`M->I->K->A->C`<br>`M->I->K->A`<br>`Deleting all nodes` |

Table 3-3. Code and Analysis for Singly Linked Lists

| Screenshot | Analysis |
|---|---|
| `//traversal`<br>`std::cout <<"Testing the DLL Traversal"<<std:: endl;`<br>`dllTraverse(head);` | The traversal function successfully displayed all the nodes in the doubly linked list from the head to the last node. This confirms that the next pointers of each node are properly connected and the list can be traversed correctly. |
| `//testing the insertion at the head`<br>`std::cout << "\nTesting the insertion at the head node"<<std:: endl;`<br>`dllInsertHead('X', &head);`<br>`dllTraverse(head);` | A new node was inserted at the beginning of the list. The new node became the head, while the previous head was moved to the next position. The links between the nodes were updated correctly. |
| `//testing the insertion at the end`<br>`std::cout << "\nTesting the insertion at the end" <<std:: endl;`<br>`dllInsertEnd('0', head);`<br>`dllTraverse(head);` | A new node was successfully inserted at the end of the list. The last node's next pointer and the new node's previous pointer were updated correctly, maintaining the structure of the doubly linked list. |
| `//General Insert`<br>`std::cout << "\nTesting the general insertion" <<std:: endl;`<br>`dllGeneralInsert('Z', second);`<br>`dllTraverse(head);` | A new node was inserted at the specified position in the list. The nodes before and after the inserted node remained connected, showing that the insertion function works properly for any valid position. |

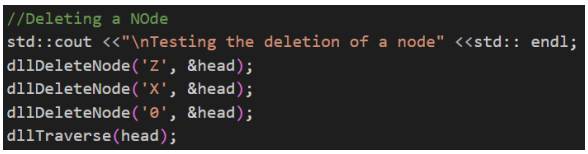
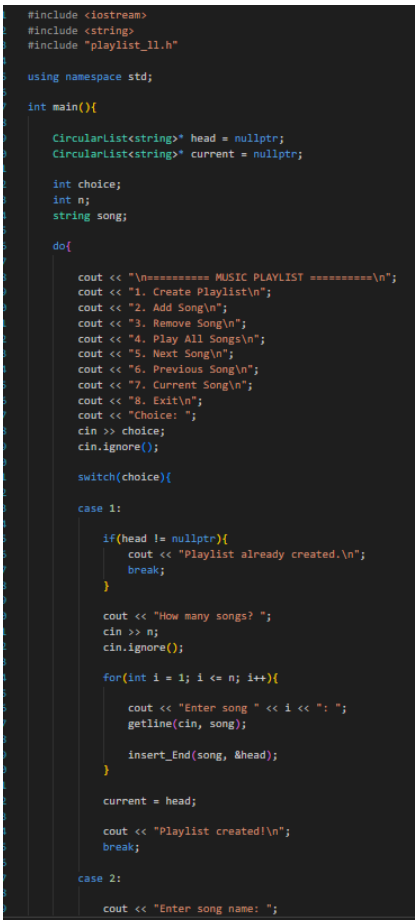
| | |
|---|---|
|  | The selected node was successfully removed from the list. The previous and next nodes were reconnected correctly, and the traversal output confirmed that the deleted node no longer existed in the list while the remaining nodes stayed in the correct order. |

Table 3-4. Modified Operations for Doubly Linked Lists

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY



```
#include <iostream>
#include <string>
#include "playlist_ll.h"

using namespace std;

int main(){

    CircularList<string>* head = nullptr;
    CircularList<string>* current = nullptr;

    int choice;
    int n;
    string song;

    do{

        cout << "\n========== MUSIC PLAYLIST ==========\n";
        cout << "1. Create Playlist\n";
        cout << "2. Add Song\n";
        cout << "3. Remove Song\n";
        cout << "4. Play All Songs\n";
        cout << "5. Next Song\n";
        cout << "6. Previous Song\n";
        cout << "7. Current Song\n";
        cout << "8. Exit\n";
        cout << "Choice: ";
        cin >> choice;
        cin.ignore();

        switch(choice){

        case 1:

            if(head != nullptr){
                cout << "Playlist already created.\n";
                break;
            }

            cout << "How many songs? ";
            cin >> n;
            cin.ignore();

            for(int i = 1; i <= n; i++){

                cout << "Enter song " << i << ": ";
                getline(cin, song);

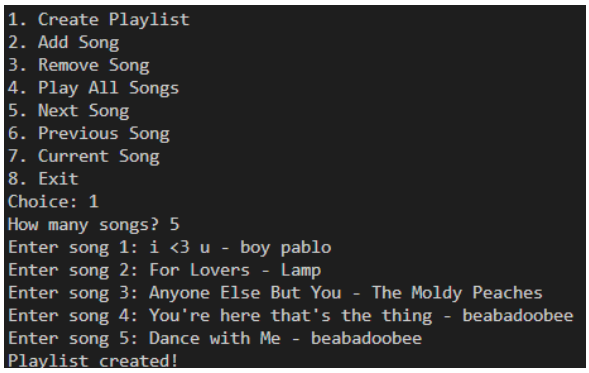
                insert_End(song, &head);
            }

            current = head;

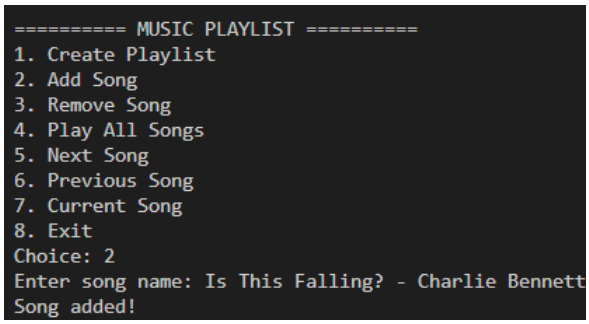
            cout << "Playlist created!\n";
            break;

        case 2:

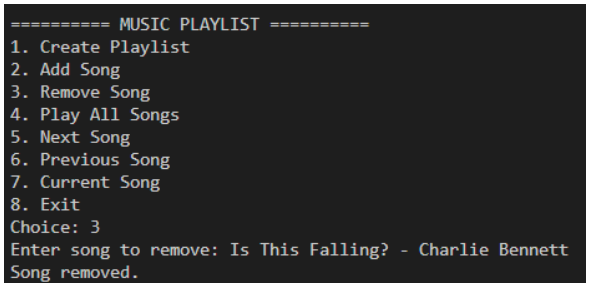
            cout << "Enter song name: ";
```



```
1. Create Playlist
2. Add Song
3. Remove Song
4. Play All Songs
5. Next Song
6. Previous Song
7. Current Song
8. Exit
Choice: 1
How many songs? 5
Enter song 1: i <3 u - boy pablo
Enter song 2: For Lovers - Lamp
Enter song 3: Anyone Else But You - The Moldy Peaches
Enter song 4: You're here that's the thing - beabadoobee
Enter song 5: Dance with Me - beabadoobee
Playlist created!
```



```
========== MUSIC PLAYLIST ==========
1. Create Playlist
2. Add Song
3. Remove Song
4. Play All Songs
5. Next Song
6. Previous Song
7. Current Song
8. Exit
Choice: 2
Enter song name: Is This Falling? - Charlie Bennett
Song added!
```



```
========== MUSIC PLAYLIST ==========
1. Create Playlist
2. Add Song
3. Remove Song
4. Play All Songs
5. Next Song
6. Previous Song
7. Current Song
8. Exit
Choice: 3
Enter song to remove: Is This Falling? - Charlie Bennett
Song removed.
```

```
        getline(cin, song);

        insert_End(song, &head);

        if(current == nullptr)
            current = head;

        cout << "Song added!\n";

        break;

    case 3:

        if(head == nullptr){
            cout << "Playlist is empty.\n";
            break;
        }

        cout << "Enter song to remove: ";
        getline(cin, song);

        delete_Node(song, &head, &current);

        break;

    case 4:

        play_All(head);

        break;

    case 5:

        if(current == nullptr){
            cout << "Playlist is empty.\n";
            break;
        }

        next_Song(&current);
        display_Current(current);

        break;

    case 6:

        if(current == nullptr){
            cout << "Playlist is empty.\n";
            break;
        }

        prev_Song(&current);
        display_Current(current);

        break;

    case 7:

        display_Current(current);

        break;

    case 8:

        destroy_List(&head);

        cout << "Goodbye!\n";

        break;

    default:

        cout << "Invalid choice.\n";

    }

}while(choice != 8);

return 0;
}
```

```
========== MUSIC PLAYLIST ==========
1. Create Playlist
2. Add Song
3. Remove Song
4. Play All Songs
5. Next Song
6. Previous Song
7. Current Song
8. Exit
Choice: 5
Now Playing: Anyone Else But You - The Moldy Peaches
```

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **ChatGPT**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** I used ChatGPT for code assistance, debugging, code formatting, and to better understand the concepts and functions used in implementing the linked list playlist.

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** ChatGPT was used to explain programming concepts, suggest improvements, identify errors, and provide guidance during coding.

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.